Branching and boundary tests

Learn to understand, design, build, test and explain AI systems responsibly.

Project: Python readiness portfolio

THE CONNECTED WEEK

One connected branching and boundary-test record.

Explain the chain, trace the path, diagnose an overwrite and state test limits.

Keep one habit: Keep exact rule, input, checked/skipped conditions, assignments and output/error evidence together.

MAIN LESSON · SECTION 1

Recall · Retrieve before notes

> Keep the stated task, its own code/values and support record together.

Task and evidence record

- Try the recall before opening notes. Then use a different colour or a labelled second line to correct your explanation.

- Without opening the old lesson, explain one key idea from Boolean **condition**s. Give an example and a mistake that the idea helps you avoid.

- Without opening the old lesson, explain one key idea from Values variables and types. Give an example and a mistake that the idea helps you avoid.

Boolean conditions

> **My remembered idea, example and correction**

Values variables and types

> **My remembered idea, example and correction**

MAIN LESSON · SECTION 5

W1 · Explain the procedure

> Keep the stated task, its own code/values and support record together.

Task and evidence record

- Duration chain: minutes<0→Invalid; otherwise minutes<=12→Short; else Long. -1 checks first only, 12 checks first False/second True, 13 both False. Replacing elif with independent if overwrites Invalid at -1. Separate book chain count<0 invalid, else count<5 small, else large; -2 invalid, 4 small, 5 large. All comparisons assume supplied integers; no arbitrary-input validator or real lamp operation is implemented.

- Reference for this exercise: open this week's student lesson, "Branching and boundary tests", at "See how it works" and "Follow a worked example". Use the original worked values or code named in the question; the later pictorial case is a separate example. Keep this reference beside your workbook while practising.

- Describe the purpose of if, elif and else in the worked duration sorter. Explain why print(label) is outside the indented **branch**es.

> **My reasoning and evidence**

MAIN LESSON · SECTION 6

W2 · Predict, trace and observe

> Keep the stated task, its own code/values and support record together.

Task and evidence record

- Duration chain: minutes<0→Invalid; otherwise minutes<=12→Short; else Long. -1 checks first only, 12 checks first False/second True, 13 both False. Replacing elif with independent if overwrites Invalid at -1. Separate book chain count<0 invalid, else count<5 small, else large; -2 invalid, 4 small, 5 large. All comparisons assume supplied integers; no arbitrary-input validator or real lamp operation is implemented.

- For -1, 12 and 13, record checked **condition**s, skipped **condition**s and the final label. Run each as a fresh script after predicting.

My reasoning and evidence

MAIN LESSON · SECTION 7

W3 · Find the first error

> Keep the stated task, its own code/values and support record together.

Task and evidence record

- Duration chain: minutes<0→Invalid; otherwise minutes<=12→Short; else Long. -1 checks first only, 12 checks first False/second True, 13 both False. Replacing elif with independent if overwrites Invalid at -1. Separate book chain count<0 invalid, else count<5 small, else large; -2 invalid, 4 small, 5 large. All comparisons assume supplied integers; no arbitrary-input validator or real lamp operation is implemented.

- Trace the two-independent-if example for -1. Identify which assignment overwrites the earlier label and make the smallest repair.

My reasoning and evidence

MAIN LESSON · SECTION 9

W5 · State an honest limit

> Keep the stated task, its own code/values and support record together.

Task and evidence record

- Duration chain: minutes<0→Invalid; otherwise minutes<=12→Short; else Long. -1 checks first only, 12 checks first False/second True, 13 both False. Replacing elif with independent if overwrites Invalid at -1. Separate book chain count<0 invalid, else count<5 small, else large; -2 invalid, 4 small, 5 large. All comparisons assume supplied integers; no arbitrary-input validator or real lamp operation is implemented.

- Reference for this exercise: open this week's student lesson, "Branching and boundary tests", at "See how it works" and "Follow a worked example". Use the original worked values or code named in the question; the later pictorial case is a separate example. Keep this reference beside your workbook while practising.

- A repair changes the small-count **condition** from < 6 to <= 6. Name the case that exposes the original error and two different kinds of earlier cases to rerun.

> **My reasoning and evidence**

MAIN LESSON · SECTION 4

Try alone · Selected independent task

> Keep the stated task, its own code/values and support record together.

Task and evidence record

- Predict the action for 30, then trace 29 and -1. Explain why validity is checked before the lamp decision.

- Attempt this case before feedback. Record any help. Earlier examples below are further practice; a case already practised with feedback cannot establish fresh transfer.

Independent case: Trace branch order

```
reading = 30
if not 0 <= reading <= 100:
    action = "invalid"
elif reading < 30:
    action = "on"
else:
    action = "off"
print(action)
```

> **My independent explanation, trace and limits**
>
> | |
> |---|
> | |
> | |
> | |
> | |

MAIN LESSON · SECTION 10

Project · Keep code and evidence

> Keep the stated task, its own code/values and support record together.

Task and evidence record

- Add a labelled **branch**ing and **boundary** tests **evidence** sheet to Python readiness portfolio. Show how you can trace the selected **branch** in an if elif else chain.

  Use the supplied fictional case and keep its **input**s, intermediate steps, calculation or **prediction**, and limitation. Connect this record to last week's record; a matching answer without a trace is insufficient.

> **My project change, supporting evidence and remaining limit**

MAIN LESSON · SECTION 11

Feedback · Repair and return

> Keep the stated task, its own code/values and support record together.

Task and evidence record

- Revisit this idea with your teacher after a delay. Familiarity is different from explaining and using it on a changed case.

Evidence	My record
Worked alone / hint / model	
First step I repaired and why	
New case checked later; date and result	

Evidence	My record
What I can now explain without notes	
One remaining question	

Reference bank and extra practice

Choose the reference that helps your next step.

▼ **Reference · Supported W2 record**

REFERENCE · SECTION 2

Reference · Supported W2 record

> Keep the stated task, its own code/values and support record together.

Task and evidence record

- Use W2: Trace the tested path for **Branch**ing and **boundary** tests. Record the answer once; the lesson and workbook refer to the same attempt.

Given inputs	Procedure I chose	Middle steps	Result and check

The step I completed without help, and the hint I needed

▼ Extra practice · Individual reasoning check

REFERENCE · SECTION 3

Extra practice · Individual reasoning check

> Keep the stated task, its own code/values and support record together.

Task and evidence record

- Use the worked case for **Branch**ing and **boundary** tests. Proposed explanation: “Every **condition** in the chain always runs”. Decide whether this explanation is supported.

  Point to the step or supplied fact that decides; explain your repair if needed.

> **My explanation before partner discussion**

▼ Additional practice · Earlier W4 transfer

REFERENCE · SECTION 8

Additional practice · Earlier W4 transfer

> Keep the stated task, its own code/values and support record together.

Task and evidence record

- Use an integer tiles count. Label a negative value Invalid, zero Empty, 1 through 6 Small and values above 6 Large. Write an ordered if elif else script and predict -1, 0, 1, 6 and 7. Include one print after the chain.

My reasoning and evidence

▼ **Extra practice · Pictured case**

REFERENCE · SECTION 12

Extra practice · Pictured case

> Keep the stated task, its own code/values and support record together.

Task and evidence record

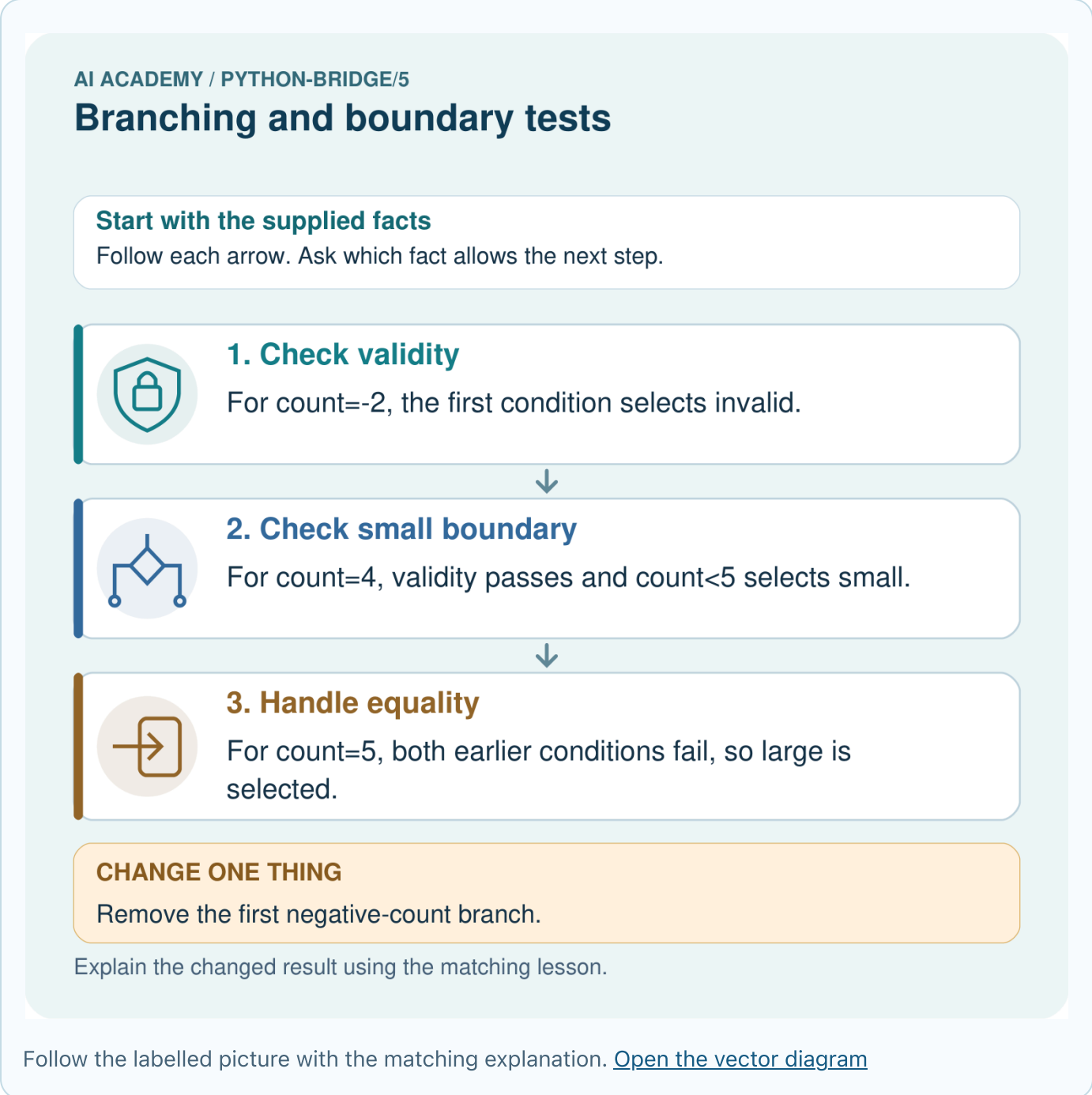
- Check invalid values before **branch**ing. A toy shelf label uses fewer than five books as "small", otherwise "large", but negative counts are invalid.

- Python 3: if count<0: label="invalid"; elif count<5: label="small"; else: label="large". Read as indented **branch**es.

- Trace -2, 4 and 5 through the **branch**es. Explain why the negative check comes first and what label -2 gets if it is removed.

- Support used: alone / hint / worked example. This record is practice **evidence**.
- Project connection: For Python readiness before the Responsible Plant Classifier, retain a trace of the values, the predicted **output**s and the changed-case explanation.

  Use actual Python 3 results to check the **prediction** after reasoning first.



Follow the labelled picture with the matching explanation. Open the vector diagram

My picture-based explanation and changed-case prediction

▼ **Reference · Project portfolio milestone**

REFERENCE · SECTION 13

Reference · Project portfolio milestone

> Keep the stated task, its own code/values and support record together.

Task and evidence record

- Project: Python readiness portfolio
- Current milestone: Python readiness portfolio. Keep runnable code, **prediction**s, actual **output**s, one repaired error and an explanation of a function on changed **input**s.
- For Python readiness before the Responsible Plant Classifier, retain a trace of the values, the predicted **output**s and the changed-case explanation. Use actual Python 3 results to check the **prediction** after reasoning first.

Evidence to keep	My record
This week's input and deciding step	
Prediction and actual result (or labelled paper trace)	
One change since the previous checkpoint and why	

Evidence to keep	My record
One error or limit and the next test	

> **Explain what this week adds to your project. Reuse last week's record rather than starting again.**

▼ **Extra practice · Predict, check and repair**

REFERENCE · SECTION 14

Extra practice · Predict, check and repair

> Use only after an attempt; keep the reserved assessment separate.

First question and hint

- Check invalid values before **branch**ing. A toy shelf label uses fewer than five books as "small", otherwise "large", but negative counts are invalid.
- Which labelled step matches this detail? For count=5, both earlier **condition**s fail, so large is selected.
- 1. Check validity
- 2. Handle equality

- 3. Check small **boundary**

- Hint if needed: Read the steps in the pictorial case. Match the supplied detail to the stage that performs or records it.

Practice 1: explain the deciding evidence

> **My selected conclusion, deciding fact and explanation**

▼ **Check the first explanation**

After your attempt, compare with the supported explanation: Handle equality For count=5, both earlier **condition**s fail, so large is selected. An if/elif/else chain selects the first matching **branch**.

Putting invalid handling first prevents a negative count from being mislabelled as a small valid collection.

Changed-case question and hint

- Changed-case practice: Remove the first negative-count **branch**. Which conclusion is supported by the supplied facts?

- 1. This one example guarantees correct results for every future **input**.

- 2. A count of -2 would incorrectly receive small.

- 3. We can decide the result without examining the changed **input** or the procedure.

- Hint if needed: Use the changed **condition** and the deciding step in the pictorial trace. The answer must say what follows from those facts.

Practice 2: explain the deciding evidence

My selected conclusion, deciding fact and explanation

▼ **Check the changed explanation**

After your attempt, compare with the supported explanation: A count of -2 would incorrectly receive small. Negative two is less than five, but it is not a permitted measured count.

Repair and keep the record

Review Boolean **condition**s (python-bridge/4). Goal: Translate stated requirements into comparisons. Short practice: Give both comparison results for ages 11, 15 and 16.

Which comparison rejects each outside value, and why does and matter?. Return to this week and explain what you changed.

Review Values variables and types (python-bridge/2). Goal: Distinguish integer, floating-point, text and Boolean values. Short practice: Predict both print **output**s and explain why they differ.

Does count+tag work, and what happens to each valid **output** after count becomes ten?. Return to this week and explain what you changed.

The browser lab lets you edit the supported Python examples already included in this lesson. Predict first, run, inspect the actual **output** or error, change one **input**, then run again.

Keep code, **prediction**s, actual results and a reasoned revision together. On paper, label your result as a trace rather than an execution.

Use Download completed work in the hosted lesson to export your answers, reflections, practice feedback and code-run **evidence**. Open the HTML copy to read or print it.

Drafts stay in this browser when storage is available; clear drafts on a shared device.

What I repaired, the evidence I kept and the next test

▼ Earlier wording, alternatives and picture transcripts

Use this supported practice once, in the browser or on paper. Keep the reserved independent assessment for a separate first attempt before teacher feedback.

Repair a missing building block

Predict, run and debug

Keep your completed work

▼ Reference · Six-picture response sheets

REFERENCE · SECTION 15

Reference · Six-picture response sheets

> Use the picture relevant to a current gap.

01. Branching and boundary tests: the case

AI ACADEMY / PYTHON-BRIDGE/5 · 01 · LOOK AT THE FACTS

Branching and boundary tests: the case

The situation

Check invalid values before branching. A toy shelf label uses fewer than five books as "small", otherwise "large", but negative counts are invalid.

Supplied evidence

Python 3: if count<0: label="invalid"; elif count<5: label="small"; else: label="large". Read as indented branches.

Trace branching and boundary tests and explain the changed program before running it.

Follow the labelled picture with the matching explanation. Open the vector diagram

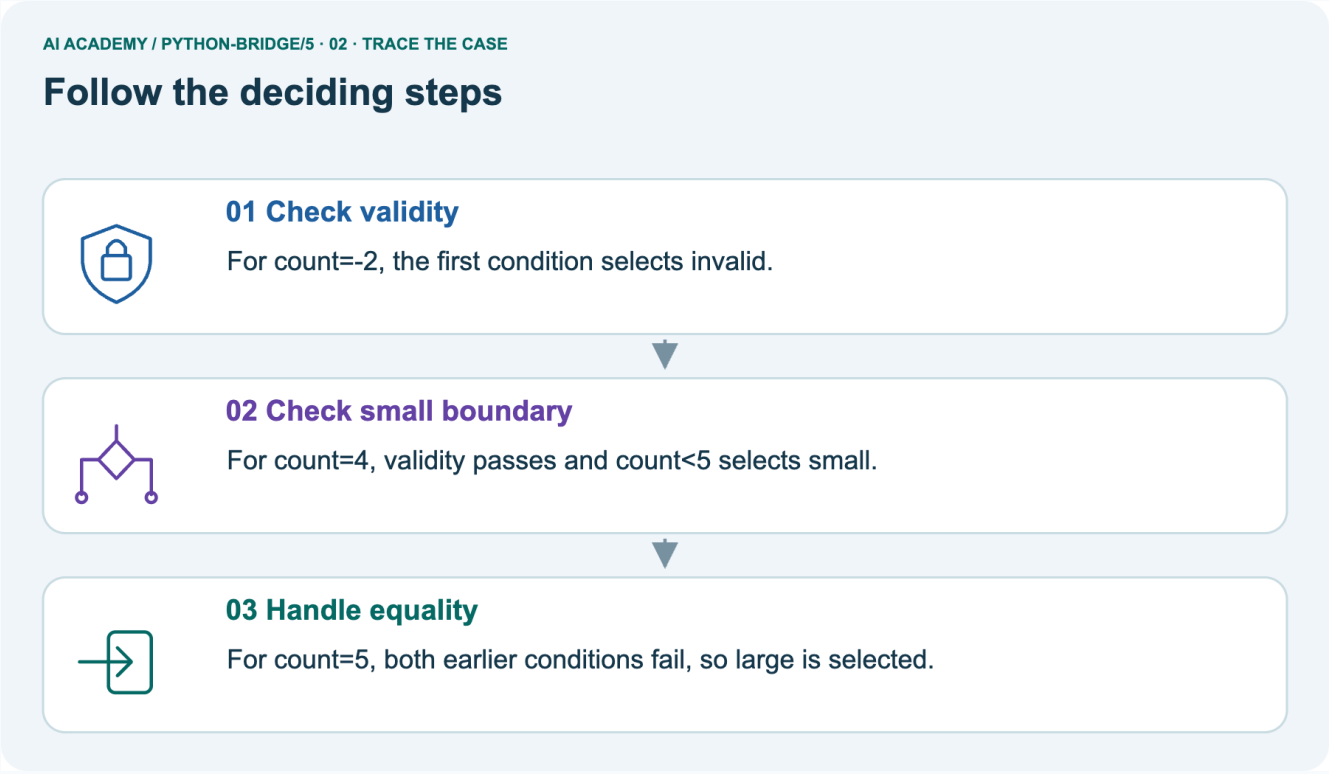
Check invalid values before **branch**ing. A toy shelf label uses fewer than five books as "small", otherwise "large", but negative counts are invalid.

Supplied facts: Python 3: if count<0: label="invalid"; elif count<5: label="small"; else: label="large". Read as indented **branch**es.

Point to the given fact that matters most. Explain why it matters.

My picture explanation, deciding evidence and one limit

02. Follow the deciding steps



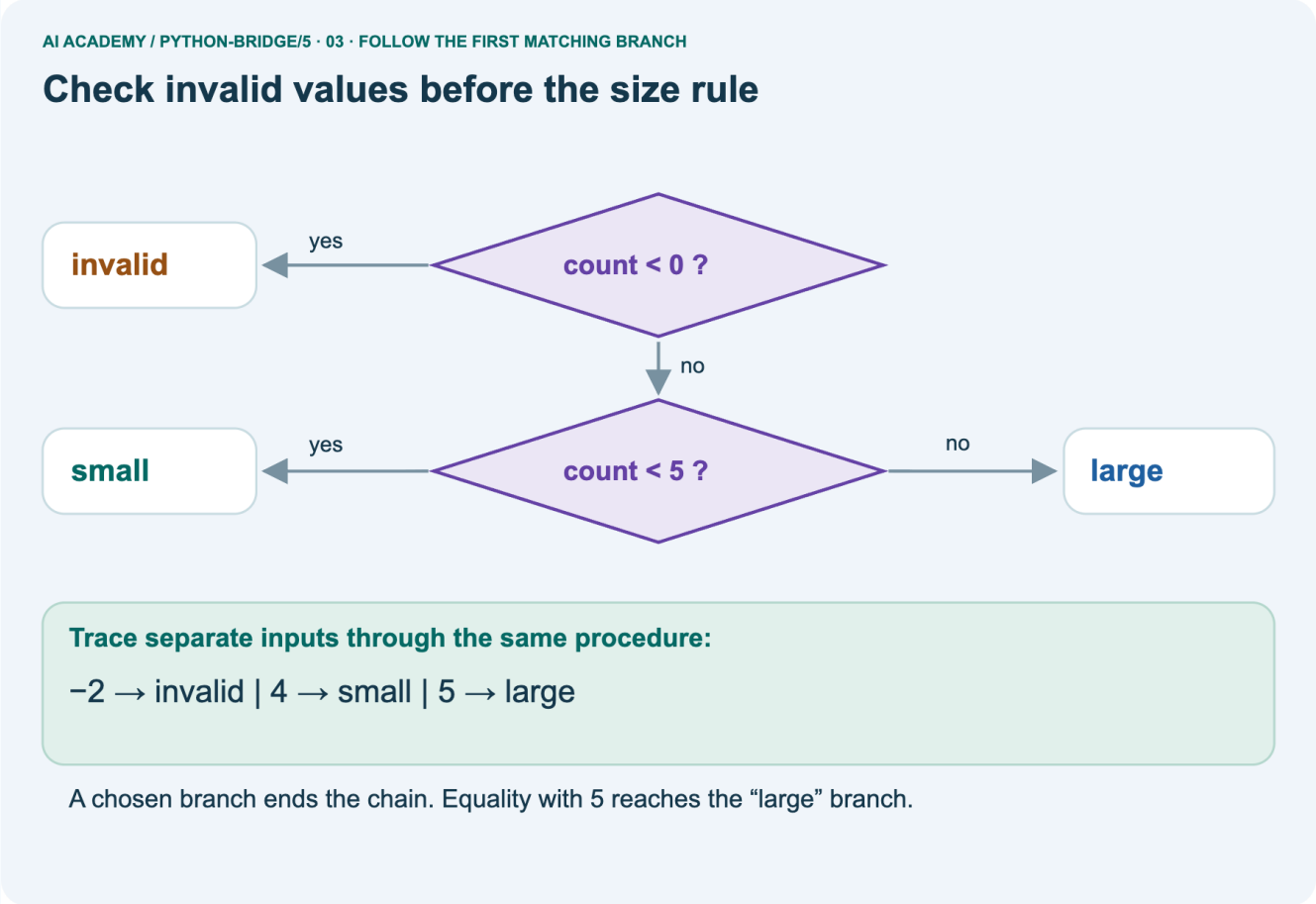
Follow the labelled picture with the matching explanation. Open the vector diagram

Check validity: For count=-2, the first **condition** selects invalid. → Check small **boundary**: For count=4, validity passes and count<5 selects small. → Handle equality: For count=5, both earlier **condition**s fail, so large is selected.

Point to each numbered stage and explain which recorded **input** it uses.

My picture explanation, deciding evidence and one limit

03. Check invalid values before the size rule



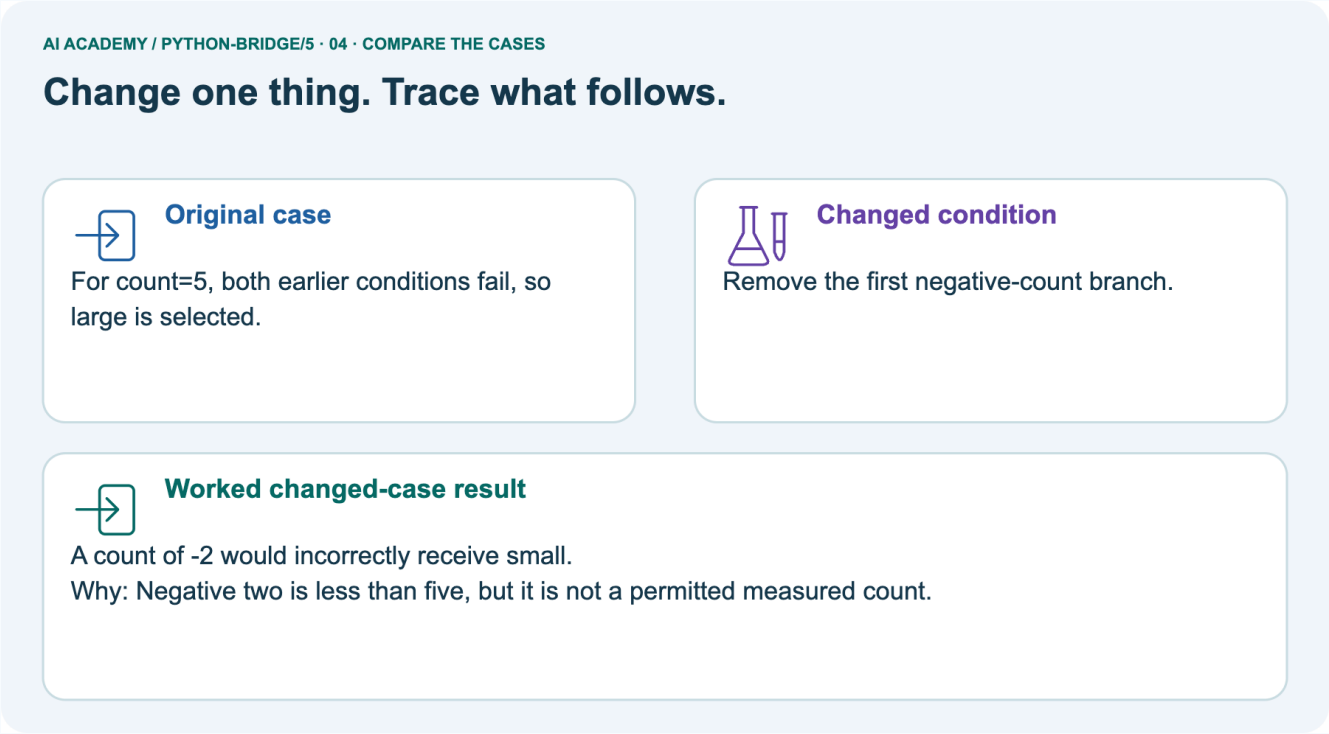
Follow the labelled picture with the matching explanation. Open the vector diagram

First ask count less than zero: yes selects invalid. Otherwise ask count less than five: yes selects small; no selects large. Separate **input** traces are minus two invalid, four small, five large.

Read the labels and values. Explain the relationship the model shows.

My picture explanation, deciding evidence and one limit

04. Change one thing. Trace what follows.



Follow the labelled picture with the matching explanation. Open the vector diagram

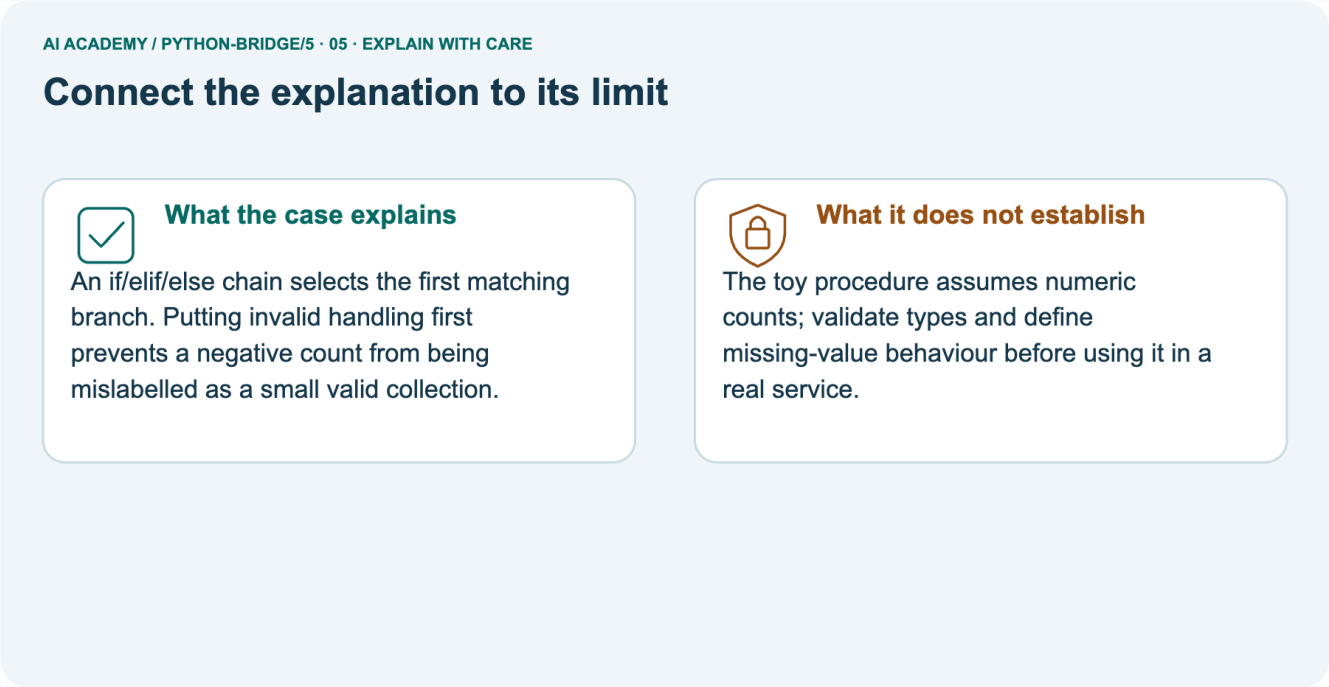
Original case: For count=5, both earlier **condition**s fail, so large is selected. Changed **condition**: Remove the first negative-count **branch**. Expected result: A count of -2 would incorrectly receive small.

Reason: Negative two is less than five, but it is not a permitted measured count.

Predict before checking the worked changed-case result. Name what changed and what the supplied facts allow you to conclude.

My picture explanation, deciding evidence and one limit

05. Connect the explanation to its limit



Follow the labelled picture with the matching explanation. Open the vector diagram

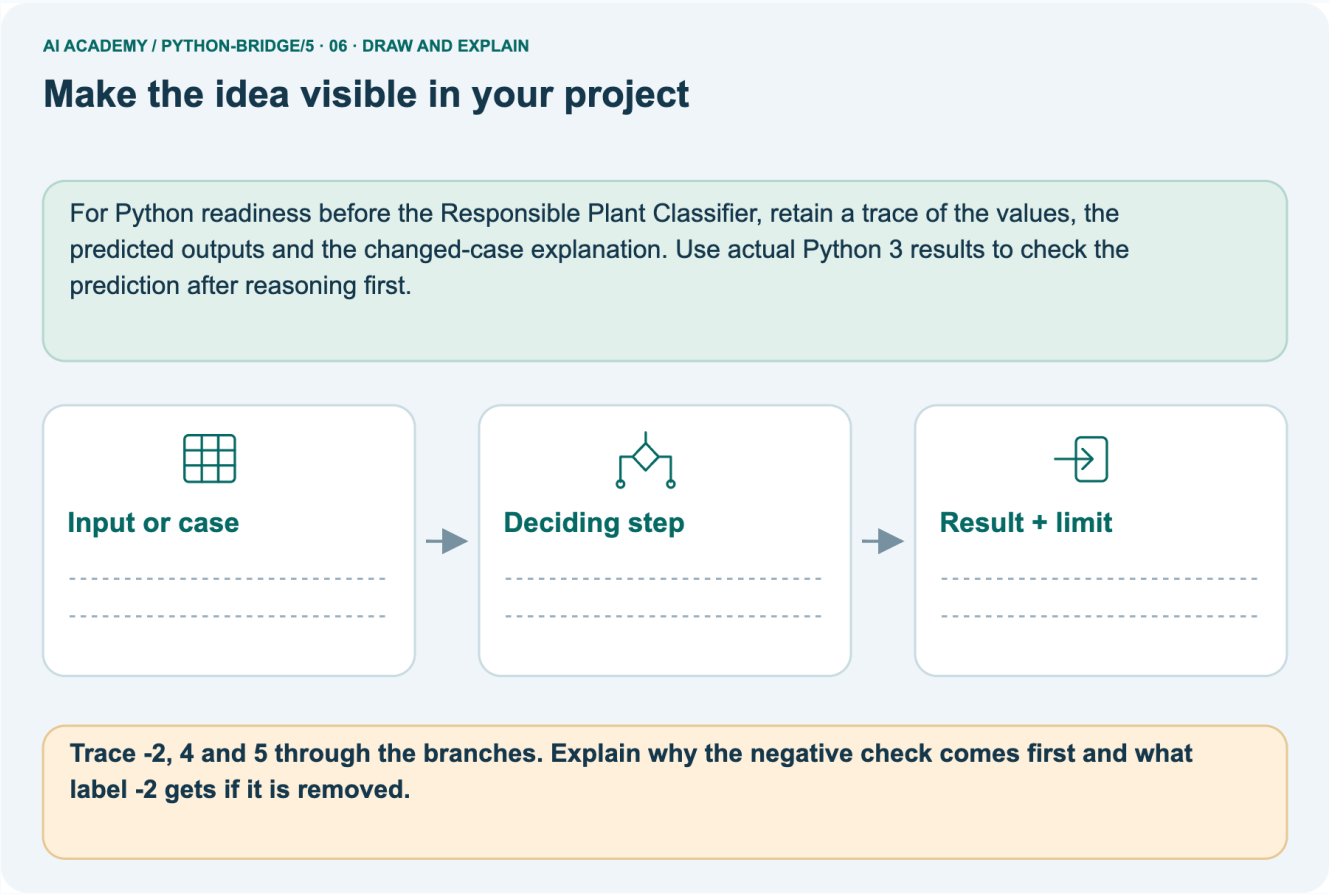
An if/elif/else chain selects the first matching **branch**. Putting invalid handling first prevents a negative count from being mislabelled as a small valid collection.

Limit: The toy procedure assumes numeric counts; validate types and define missing-value behaviour before using it in a real service.

Explain the case, then point to a limit on the claim.

My picture explanation, deciding evidence and one limit

06. Make the idea visible in your project



Follow the labelled picture with the matching explanation. Open the vector diagram

For Python readiness before the Responsible Plant Classifier, retain a trace of the values, the predicted **output**s and the changed-case explanation. Use actual Python 3 results to check the **prediction** after reasoning first.

Draw your **input**, deciding step, result and limit. Supported question: Trace -2, 4 and 5 through the **branch**es. Explain why the negative check comes first and what label -2 gets if it is removed.

Draw your project trace and justify the deciding step using **evidence**.

My picture explanation, deciding evidence and one limit

<details>
<summary>Earlier wording, alternatives and picture transcripts</summary>

01. Branching and boundary tests: the case

02. Follow the deciding steps

03. Check invalid values before the size rule

04. Change one thing. Trace what follows.

05. Connect the explanation to its limit

06. Make the idea visible in your project
</details>

<details>
<summary>Optional interactive practice and completed work</summary>

Save or print your responses for your portfolio. Progress and drafts on the hosted website stay in the current browser. Use only fictional or teacher-approved data.
</details>